Escalonamento de Processos

Professor: Paulo Roberto de Oliveira
Disciplina: Sistemas Operacionais

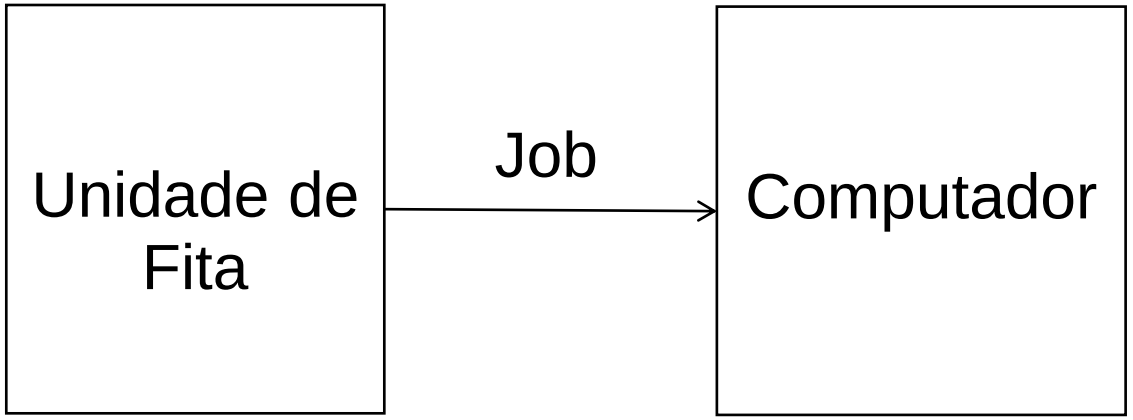
Departamento de Informática
Universidade Estadual de Maringá

Introdução

- Há situações em que mais de um processo têm condições de execução em um momento;
- Deve-se decidir qual deles vai “tomar posse” do processador;
- Quem toma essa decisão é o **Scheduler (Escalonador)**.

Introdução

- Num sistema batch, temos o algoritmos mais simples de escalonamento.



- Um job é lido da unidade de fita;
- O job é carregado na memória e executa até o fim;
- Quando um job termina, o sistema carrega outro.

Introdução

- Num sistema **timesharing multi-usuário**, o algoritmos é bem mais complexo:
  - Pode haver vários processos prontos;
  - Vários usuários esperam que suas tarefas sejam completadas;
  - Há processos rodando em background (ex.: mail).

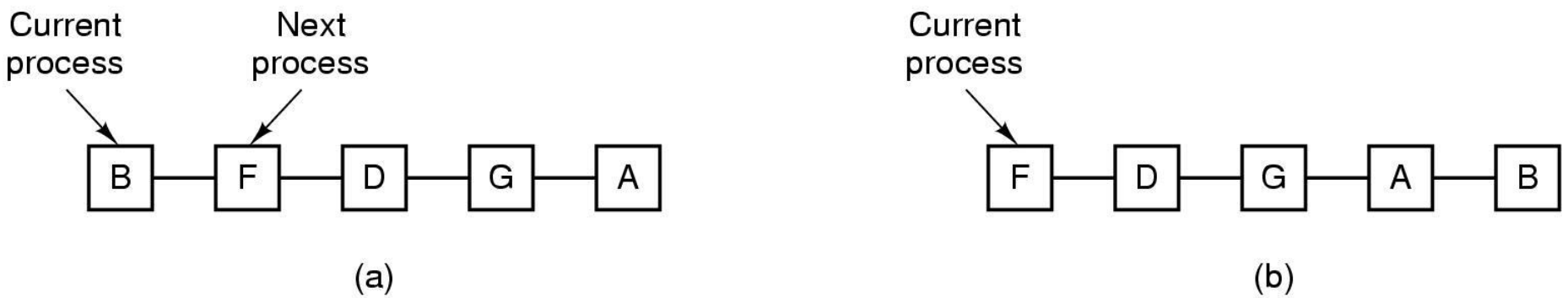
Objetivos de um Escalonador

1. **Justiça**: cada processo recebe um quantum adequado;
2. **Eficiência**: a finalidade do escalonamento é manter a CPU ocupada ~100%;
3. **Tempo de resposta**: a demora para um processo iniciar sua execução deve ser pequena;
4. **Turnaround**: o tempo que um processo “fica no sistema” deve ser o menor possível;
5. **Throughput**: alto número de jobs processados por hora;

Alguns Conceitos

- **Tick**: Período entre uma interrupção por tempo e a seguinte.
- **Preempção**: Suspensão temporária de um processo que poderia continuar executando. O processo deixa o processador e fica numa fila de processos “prontos para executar”.
- Um sistema preemptivo é o contrário de um sistema batch “rodar até o fim” (não preemptivo).

Escalonamento Round Robin



- Cada processo recebe um **quantum**: período de tempo em que ele pode utilizar o processador;
- No final do quantum, se o processo ainda estiver executando, ele sofre preempção e a CPU é entregue ao próximo processo;
- Caso o processo corrente solicite um I/O e bloqueie, a CPU é entregue ao próximo processo.

Escalonamento com Prioridades

- No método Round Robin, todos os processos são iguais.
- Funcionamento:
  - Cada processo recebe uma **prioridade**;
  - O processo “pronto” com maior prioridade é aquele que executa.
  - Para evitar que um processo execute indefinidamente, o escalonador pode <u>reduzir</u> sua prioridade a cada <u>tick</u>;
  - Quando a prioridade do processo atual ficar abaixo da prioridade de outro processo pronto, ocorre <u>preempção</u>.
  - <u>Comando nice</u>: um usuário pode reduzir a prioridade de seu processo caso ele não seja importante – “legal” para os outros usuários (ninguém usa!).

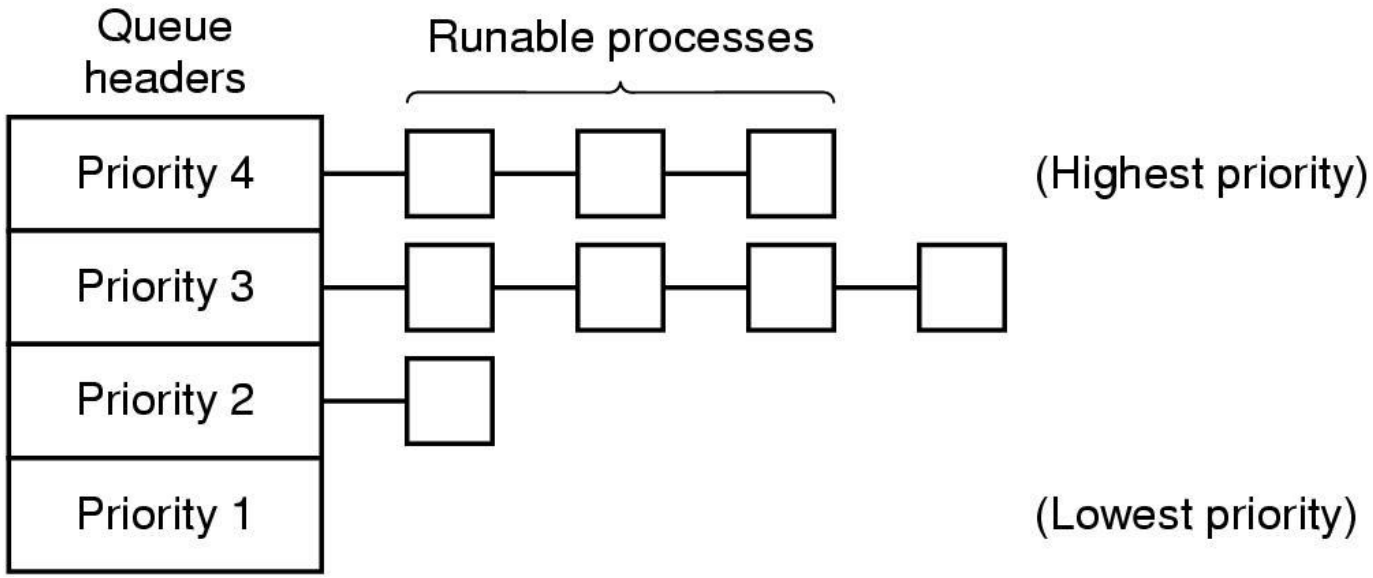
Escalonamento com Prioridades

Prioridade Dinâmica:

- Cada processo recebe uma prioridade que representa o inverso da fração utilizada de seu quantum;
- Ex.: Se ele utilizar 2 ms de um quantum de 100 ms, sua prioridade será 50;
- Ex.: Se usar 50 ms, sua prioridade será 2;
- Ex.: Se 100%, prioridade 1.

Escalonamento com Prioridades

Classes de prioridades:



- Enquanto houver processos prontos na classe 4, os processos das classes abaixo nunca serão executados.

Escalonamento com Próximo Menor Processo

- O método de Menor Job Primeiro produz o menor tempo de resposta médio para sistemas batch. Se pudéssemos usar um método desse tipo para sistemas interativos, teríamos o menor tempo de resposta para requisições.
- O problema reside em determinar qual é o tempo necessário para processar uma requisição.
- Uma forma de determinar isso é estimar baseado nos tempos passados, mas usando pesos, de forma que as medidas mais recentes sejam mais importantes.

Escalonamento com Próximo Menor Processo

- O método de Menor Job Primeiro produz o menor tempo de resposta médio para sistemas batch.
- Se pudéssemos usar um método desse tipo para sistemas interativos, teríamos o menor tempo de resposta para requisições.
- O problema reside em determinar qual é o tempo necessário para processar uma requisição.

Escalonamento com Próximo Menor Processo

- Uma forma de determinar isso é estimar baseado nos tempos passados, mas usando pesos, de forma que as medidas mais recentes sejam mais importantes.
- A técnica de estimar valores em uma série com peso, em que os valores mais recentes são mais importantes, é chamada de **aging**.

Escalonamento Garantido

- Abordagem: fazer promessas aos usuários sobre o desempenho do sistema - com n usuários, cada um tem garantido a utilização de 1/n do tempo de CPU.
- Para alcançar este objetivo, o sistema precisa saber quanta CPU cada job do usuário já consumiu desde o login e quanto tempo se passou desde o login de cada usuário.
- Com esses valores, pode-se computar a taxa de consumo de CPU por usuário.

Escalonamento Garantido

- Em seguida, o sistema pode fazer a correção da utilização dando preferência ao usuário com menor taxa de consumo.
- Uma ideia parecida pode ser aplicada em sistemas de tempo real, comparando os deadlines que cada processo precisa atender. O sistema pode dar preferência ao job com deadline de mais risco de ser perdido.

Escalonamento da Loteria

- Waldspurger e Weihl, 1994:
- Os processos recebem tickets de loteria para vários recursos, inclusive CPU;
- O escalonador sorteia um ticket ao acaso;
- O processo que possuir este ticket ganha acesso ao recurso;
- Cada ticket vale uma certa quantidade do recurso;
- Processos de mais importância podem receber tickets extras;

Escalonamento da Loteria

- **Ex.:** um período de tempo dividido em 100 tickets.
- Se um processo tiver 20 tickets, tem 20% do tempo.
- A consumação é aleatória.
- A chance de um processo ser escolhido no sorteio é proporcional ao número de tickets que ele possui;
- Processos cooperativos podem partilhar tickets;

Escalonamento da Loteria

- Ex.: um cliente faz uma requisição a um servidor e bloqueia.
- Ele empresta seus tickets para aumentar a chance do servidor ser selecionado;
- Quando o servidor responder, devolve os tickets ao cliente.

Escalonamento do Compartilhamento Justo

- Suponha que 2 usuários de um sistema tem a mesma prioridade. O usuário A executa 9 processos e o usuário B apenas 1.
- Sem outras informações, supondo que os processos sejam todos semelhantes, o usuário A vai obter 90% do uso da CPU, enquanto B vai ficar com os 10% restantes.
- Para evitar esta situação, alguns sistemas verificam a quem pertence um processo quando a CPU é alocada.

Escalonamento do Compartilhamento Justo

- Considere um usuário 1 com 4 processos: A, B, C e D; e um usuário 2 com o processo E.
- Um sistema Round Robin que use este método poderia produzir a seguinte sequência de alocação:
- A E B E C E D E A E B E C E D E ...
- Caso haja interesse em dar o dobro de CPU de B para A, a sequência seria:
- A B E C D E A B E C D E ...

Escalonamento de Tempo Real

- Casos em que uma resposta atrasada é tão ruim quanto nenhuma.
- Exemplos:
  - Monitoramento de pacientes;
  - Piloto automático
  - Controle de um reator nuclear.

Escalonamento de Tempo Real

- Tipos:
  - Hard real time: Deadlines rigorosos;
  - Soft real time: Perder algum deadline é aceitável;
  - Eventos periódicos ou aperiódicos.

Escalonamento de Tempo Real

- **Ex.:** Um sistema possui m eventos periódicos; o evento i ocorre no período Pi e necessita de Ci segundos de tratamento; a carga só pode ser manipulada se o sistema atender à regra:

$$\sum_{i=1}^{m} \frac{C_i}{P_i} \leq 1$$

- Um sistema de tempo real que atende este critério é chamado de **escalonável**.

Escalonamento de Tempo Real

Exemplo:

- 3 eventos com períodos de 100, 200 e 500 mseg (Pi).
- Tratamento em 50, 30 e 100 mseg, consecutivamente (Ci).
- Escalonável: 0,5 + 0,15 + 0,2 < 1.

Escalonamento de Tempo Real
Algoritmos

1) Taxa monotônica

- Liu e Layland, 1973, clássico.
- Prioridade proporcional à frequência.
- Faz preempção de qualquer processo para rodar o mais alto.
- Algoritmo ótimo!

Escalonamento de Tempo Real
Algoritmos

2) Primeiro deadline primeiro:

- Processos que tratam eventos ficam na fila de prontos.
- A fila é classificada por deadline.

3) Menor margem:

- Ex.: Tratamento de 200 mseg, deadline de 250 mseg, margem de 50 mseg

Escalonamento de Threads

- Num sistema de múltiplos processos, que admite múltiplas threads, há 2 níveis de paralelismo: processos e threads.
- Quando o sistema implementa threads no nível do usuário, o kernel não toma conhecimento de sua existência. Ele continua escalonando apenas processos. O escalonador interno de cada processo decide qual thread deve executar.
- Como não há interrupção por tempo para threads, a thread selecionada pode executar indefinidamente. Eventualmente ela usará todo o quantum do processo. Nesse caso, o kernel selecionará outro processo para execução.

Escalonamento de Threads

- Quando esse processo retomar a CPU, aquela thread continuará sua execução até que eventualmente termine.
- Porém, seu comportamento “anti-social” não afetará outros processos. O escalonador do kernel dará a cada um o tempo que ele achar justo, independente do que acontece dentro daquele processo.

Escalonamento de Threads

- Por outro lado, quando as threads são implementadas no kernel, ele escolhe que thread vai executar em determinado momento, independente do processo a que pertence. É claro que, se houver interesse, o kernel pode considerar esse dado para tomar sua decisão.
- A principal diferença entre threads no nível do usuário e no kernel é relativo ao desempenho:
  - Fazer uma troca de threads num sistema de threads no nível do usuário significa gastar apenas algumas instruções;

Escalonamento de Threads

- No nível do kernel, significa uma troca de contexto completa (se o processo for diferente), mudar o mapa de memória e invalidar o cache, o que é várias ordens de magnitude mais lento;
- Por outro lado, num sistema no nível do kernel o I/O de uma thread não bloqueia as demais.

- Como o kernel sabe que o custo para a troca de contexto é muito alto, ele pode usar essa informação e escolher uma thread do mesmo processo em vez de uma thread de um processo diferente (desde que seja possível e interessante).

Escalonamento de Threads

- Uma vantagem para threads no nível do usuário é que o escalonador interno do processo pode escolher as threads pelo que elas fazem (ele sabe!). No nível do kernel, o máximo que é possível fazer é dar mais prioridade a uma delas, que será escolhida por causa disso.

Dúvidas?